Amrita School of Computing

Amritapuri Campus

Department of Computer Science and Engineering

B.Tech Computer Science and Engineering

23CSE498 – Project Phase II

Design and Solution Submission

Project Title Runtime Computational Memory: Understanding Emergent Behaviour through KV-Cache Dynamics in Transformer Language Models

Project Guide Thara Shyam, Department of Computer Science and Engineering

Team Members

Team No.	Roll Number	Name of Student
D10	AM.SC.U4CSE23352	S Kunal Achintya Reddy
D10	AM.SC.U4CSE23358	V Sriman Vashishta
D10	AM.SC.U4CSE23330	Raaghavan M S
D10	AM.SC.U4CSE23311	B Nithilesh

Academic Year: 2026–2027

Amrita Vishwa Vidyapeetham

1 Sustainable Development Goal (SDG) Mapping

Primary SDG: SDG 10 – Reduced Inequalities

Secondary SDG(s) : SDG 9 – Industry, Innovation and Infrastructure; SDG 16 – Peace, Justice and Strong Institutions

Justification :

Large language models increasingly mediate hiring shortlists, healthcare triage, educational feedback and public-service chatbots. When these systems carry social bias, the harm falls disproportionately on groups already disadvantaged along gender, religion, disability and socio-economic lines — precisely the inequalities SDG 10 targets. Current mitigation depends on retraining or fine-tuning, which is expensive, opaque, and out of reach for the smaller institutions and low-resource deployments that most need it. This project shows that a measurable part of bias amplification lives in the Key–Value cache, a runtime structure that can be observed and edited during inference at negligible cost and with no parameter updates. By releasing an open instrumentation framework, an annotated trace dataset and a causal intervention toolkit, the work makes bias auditing and correction affordable and reproducible — advancing SDG 10, while supporting SDG 9 through open and trustworthy AI infrastructure and SDG 16 through accountable, transparent automated decision-making.

2 Project Milestones and Timeline

Semester	Major Activity / Milestone	Expected Deliverable	Expected Completion	Status
S6	Project domain exploration	Problem domain identified	15 Feb 2026	Completed
S6	Literature review	Initial literature survey	15 Mar 2026	Completed
S6	Baseline implementation	Reference implementation completed	30 Apr 2026	Completed
S7 (Submission 1)	Literature survey	Comprehensive literature survey	15 Jul 2026	Completed
S7 (Submission 1)	Research gap identification	Research gap statement	20 Jul 2026	Completed
S7 (Submission 1)	Problem statement	Finalized objectives	25 Jul 2026	Completed
S7 (Submission 2)	System design	Architecture and workflow	06 Aug 2026	Current
S7	Prototype implementation	Functional prototype	15 Oct 2026	Planned
S7	Module integration	Integrated system	15 Nov 2026	Planned
S7	Testing and validation	Experimental results	15 Dec 2026	Planned
S8	System enhancement	Improved system	31 Jan 2027	Planned
S8	Performance evaluation	Performance analysis	28 Feb 2027	Planned
S8	Research publication	Paper submission	20 Mar 2027	Planned
S8	Dissertation writing	Final report	10 Apr 2027	Planned
S8	Final demonstration	Presentation and viva	20 Apr 2027	Planned

Table 1: Overall Project Timeline

3 System Architecture (If Applicable)

Architecture Description

The system is organised as six layers, each owning one stage of the pipeline from probe seeding to causal validation.

The **Experiment Control Layer** defines what is run. It draws seeds and bias probes

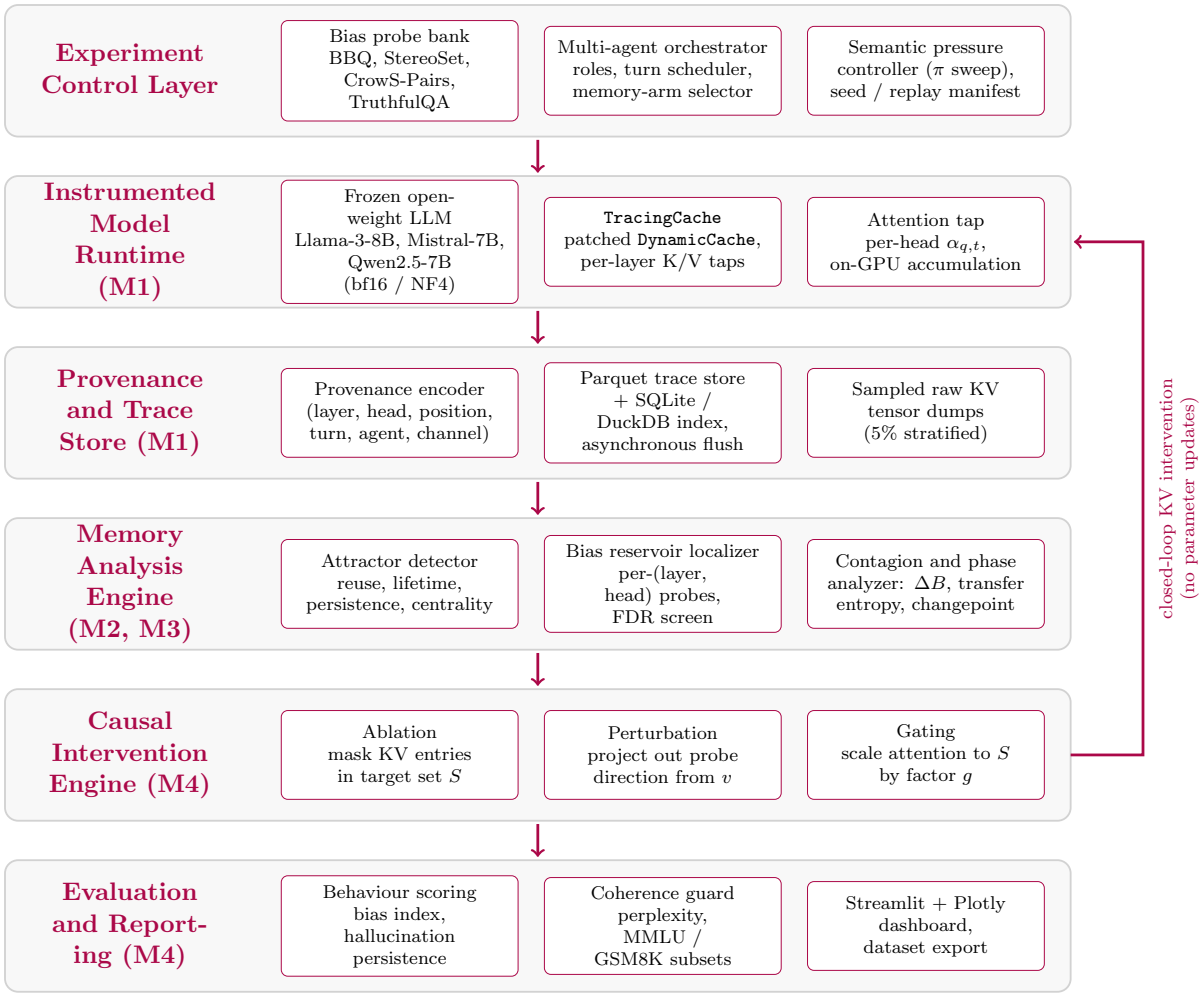


Figure 1: Layered system architecture. Data flows top-down from probe seeding to evaluation; the Causal Intervention Engine closes the loop back into the model runtime, editing only the KV cache during a seed-matched replay.

from BBQ, StereoSet, CrowS-Pairs and TruthfulQA, assigns agent roles, and sets two experimental knobs: the *memory arm* (shared-KV, text-only, or isolated control) and the *semantic pressure* π, the fraction of turns carrying stereotype-consistent content. Every run is fully described by a configuration manifest and a fixed RNG seed, so any conversation can be replayed token-for-token.

The **Instrumented Model Runtime** hosts a frozen open-weight model. No weights are modified anywhere in this system. A `TracingCache` subclasses the Hugging Face `DynamicCache` so that every write to the Key–Value store is intercepted, and forward hooks on each attention module expose the per-head attention distribution $\alpha_{q,t}^{\ell,h}$. Because fused kernels such as FlashAttention and SDPA do not materialise attention weights, the model runs in eager attention mode whenever weights must be captured, and falls back to the fused path on turns where only KV norms are logged.

The **Provenance and Trace Store** is what makes the rest of the project possible. Each

cache entry is stamped with layer, head, absolute position, turn index, agent identity and source channel (system prompt, probe, or agent-generated), and its received attention mass is accumulated on-GPU and flushed asynchronously. Storing every raw KV tensor is infeasible — a Llama-3-8B cache costs roughly 130 KB per token in bf16, so a single 4096-token conversation exceeds 500 MB — so the store keeps provenanced *statistics* for all runs and raw tensors only for a stratified 5% sample.

The **Memory Analysis Engine** consumes traces offline. It computes the attractor metrics, builds the token-level attention graph, localizes bias reservoirs through per-head probes, and runs the contagion and phase-transition analyses across memory arms and pressure levels.

The **Causal Intervention Engine** is the only layer that writes back into the runtime. Given a target set of KV entries, it applies ablation, perturbation or gating through the same hooks used for logging, during a seed-matched replay of an already-recorded conversation. This closed loop is what converts correlational observation into a causal claim.

The **Evaluation and Reporting** layer scores the replayed conversation on bias and hallucination, and enforces a coherence budget so that a bias reduction obtained merely by degrading the model is rejected rather than reported as a success.

4 Module Description

Module	Description
Module 1 Runtime Memory Instrumentation and Provenance Logging	Wraps a frozen open-weight LLM with a `TracingCache` that subclasses the Hugging Face `DynamicCache`, plus per-layer attention hooks exposing $\alpha_{q,t}^{\ell,h}$ in eager attention mode. Every cache entry is stamped with full provenance — layer, head, absolute position, turn index, agent identity and source channel — and its received attention mass is accumulated on-GPU and flushed asynchronously to a Parquet store with a SQLite index. Raw KV tensors are dumped only for a stratified sample. Also provides the multi-agent conversation harness and the three memory arms (shared-KV, text-only exchange, isolated control). **Realizes Objective 1.**

Module	Description
Module 2 Memory Structure Analysis: Attractors and Bias Reservoirs	Computes the four attractor metrics — KV reuse score, memory lifetime, attention persistence and attention-graph centrality — and flags positions exceeding all three thresholds jointly, reporting known attention sinks (early positions) as a separate baseline class so that discovered attractors are not trivially the sink phenomenon. For localization it pools value vectors of attractor positions per (layer, head), fits cross-validated logistic probes against conversation-level bias polarity, and combines probe AUC with a measured ablation effect into a reservoir score, screened for multiple comparisons against a random-head null. **Realizes Objectives 2 and 3.**
Module 3 Multi-Agent Contagion and Phase-Transition Analysis	Runs the three-arm multi-agent protocol under a swept semantic pressure π and separates the memory-mediated component of bias transfer (shared-KV arm minus text-only arm) from the text-mediated component (text-only minus isolated control), so *Memory Contagion* is defined by construction as the residual not explainable by exchanged text. Adds transfer entropy between the agents' per-turn bias series and attractor-set overlap. Fits bias against pressure with a logistic model, tests it against a linear null by AIC/BIC, and locates the critical pressure with changepoint detection and bootstrap confidence intervals, requiring agreement across seeds and models before a transition is claimed. **Realizes Objective 4.**
Module 4 Causal Intervention, Evaluation and Dashboard	Implements the three KV-only interventions — ablation (masking target entries pre-softmax), perturbation (projecting the probe-identified bias direction out of target value vectors) and gating (scaling attention to targets) — applied through the Module 1 hooks during a seed-matched replay, with no parameter updates anywhere. Measures causal effect on the bias index and on hallucination persistence, and enforces a coherence budget using perplexity and MMLU/GSM8K subsets. Includes negative controls (random and sink-only head sets) and the Streamlit/-Plotly dashboard for attention-flow, attractor-lifetime and reservoir-map visualization. **Realizes Objective 5.**

5 System Workflow

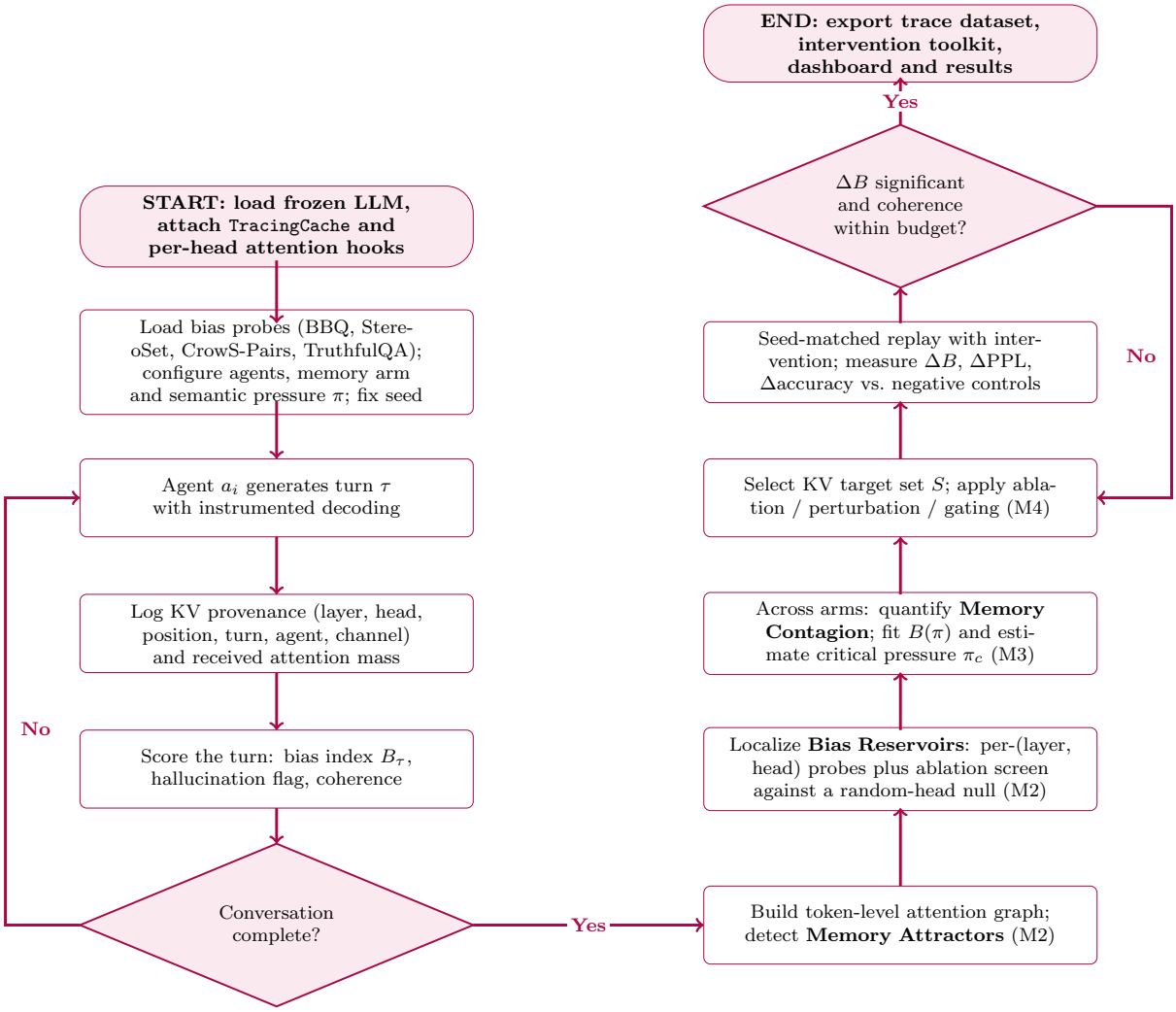


Figure 2: End-to-end system workflow. The left branch is the online instrumented conversation loop; the right branch is the offline analysis and causal validation loop, which iterates on the target set until a KV-only intervention produces a significant behavioural change within the coherence budget.

Workflow Description

Execution proceeds in two loops. The **online loop** (left branch) runs one instrumented conversation. After loading a frozen model and attaching the tracing cache and attention hooks, the orchestrator seeds the conversation from a bias probe, fixes the memory arm and the semantic pressure π, and pins every source of randomness. Each turn is then generated by one agent under instrumentation; the provenance record and attention mass for every newly written KV entry are logged, and the turn is scored for bias, hallucination and coherence. The loop repeats until the configured turn budget is exhausted. In the

shared-KV arm, the receiving agent's cache is extended with re-indexed segments of the sending agent's cache before generation; in the text-only arm only the generated text crosses between agents; in the control arm the agents never interact.

The **offline loop** (right branch) turns those traces into claims. The attention graph is built and Memory Attractors are detected; value vectors at attractor positions are pooled per head and probed to localize Bias Reservoirs; the three arms and the pressure sweep are compared to isolate Memory Contagion and to fit the bias-versus-pressure curve. A target set S of KV entries is then chosen from the reservoir ranking, and the recorded conversation is replayed under the same seed with ablation, perturbation or gating applied to S alone. If the measured effect is not statistically separable from the negative controls, or if the coherence budget is violated, control returns to target selection and S is refined. Only when both conditions hold is the causal result recorded, and the traces, toolkit, dashboard and analysis exported.

Two properties of this workflow matter for validity. First, replay is *seed-matched*: the intervened and baseline runs differ only in the KV edit, so the difference in behaviour is attributable to that edit. Second, the coherence guard is a rejection criterion, not a reported metric — an intervention that lowers bias by degrading fluency fails the workflow rather than passing it.

6 Domain-Specific Design

Design Component	Details / Reference
Database Schema / ER Diagram	Five-relation trace store: a SQLite/DuckDB index over a Parquet columnar payload. `conversation`(convId PK, model, arm, probeSuite, pressure, seed, nTurns, gitCommit); `turn`(convId FK, turnIdx, agentId, role, tokenStart, tokenEnd, biasScore, ppl); `kvRecord`(convId FK, layer, head, position, turnIdx, agentId, channel, attnMassRecv, reuseScore, lifetime, persistence, centrality, kNorm, vNorm) — the fact table, partitioned by (model, convId); `attnEdge`(convId, layer, head, qPos, kPos, weight), retained only for sampled layers; `intervention`(runId PK, convId FK, mode, targetSet, epsilon, deltaBias, deltaPpl, deltaAcc, pValue). Cardinalities: conversation 1:N turn, conversation 1:N kvRecord, turn 1:N kvRecord, conversation 1:N intervention.

Design Component	Details / Reference
UML Diagrams	*Class diagram* of the core library: `ModelAdapter` (abstract, with `LlamaAdapter`, `MistralAdapter`, `QwenAdapter` concretes), `TracingCache` (extends `DynamicCache`), `ProvenanceLogger`, `AgentRuntime`, `ConversationOrchestrator`, `MetricEngine` (composed of `AttractorDetector`, `ReservoirLocalizer`, `ContagionAnalyzer`), and `InterventionHook` realised as a Strategy pattern over `Ablate`, `Perturb`, `Gate`. *Sequence diagram* for a single turn: Orchestrator → AgentRuntime → ModelAdapter → TracingCache → ProvenanceLogger → MetricEngine. *Use-case diagram* for the researcher-facing dashboard.
Machine Learning Model Architecture	No model is trained; frozen decoder-only Transformers are the *object of study*. Llama-3-8B-Instruct: $L = 32$ layers, 32 query heads, 8 KV heads (GQA), head dimension 128, RoPE, 8k context. Mistral-7B-Instruct-v0.3: $L = 32$, 32/8 heads, sliding-window attention. Qwen2.5-7B-Instruct: $L = 28$, 28/4 heads. The only learned components are diagnostics: an L_2-regularized logistic probe per (layer, head) over attractor-pooled value vectors (input dimension 128, roughly 4k parameters per head), and the logistic/changepoint models used for the phase analysis. Attention runs in eager mode wherever weights must be materialised, since FlashAttention and SDPA do not expose $\alpha_{q,t}$.
Dataset Description	*Consumed:* BBQ — 58,492 hand-built QA instances over 9 social-bias categories in ambiguous and disambiguated contexts, supplying seeds and the primary bias index; StereoSet — intersentence and intrasentence stereotype pairs over gender, profession, race and religion, supplying the stereotype score; CrowS-Pairs — 1,508 minimal sentence pairs used as an independent probe; TruthfulQA — 817 questions in 38 categories for the hallucination-persistence case study. *Produced:* the project's own deliverable corpus, targeting $\geq$ 1,000 conversations $\times$ 3 models $\times$ 3 memory arms $\times$ 20 turns, stored as provenanced KV statistics for all runs and raw KV tensors for a 5% stratified sample.

Design Component	Details / Reference
API Design	Two surfaces. *(i) Python API:* `rcm.run(model, probes, arm, pressure, seed)`, `rcm.detect_attractors(trace)`, `rcm.localize_reservoirs(traces, labels)`, `rcm.intervene(trace, targets, mode, eps)`. *(ii) Read-only REST service* (FastAPI) backing the dashboard: `GET /conversations`, `GET /conversations/{id}/attractors`, `GET /heads/reservoir-ranking`, `GET /contagion?arm=`, and a queued `POST /replay` for intervention runs. JSON responses, auto-generated OpenAPI schema, keyset pagination over the `kvRecord` fact table.
Network Topology	Single-node and non-distributed by design. One GPU host — an institute cluster node scheduled through SLURM, or a local workstation — runs generation and writes traces to local NVMe scratch; analysis and the dashboard run on the same host or on a developer workstation mounting the trace directory read-only. The dashboard binds to `127.0.0.1` and is reached over an SSH tunnel; nothing is exposed to the campus network. Outbound access is needed only to fetch model weights and benchmark datasets from Hugging Face.
Security Architecture	The threat model is research integrity rather than adversarial attack. *Supply chain:* pinned and hash-verified dependencies, checksum-verified model downloads, `pip-audit` in CI. *Access:* no public listener, localhost-bound dashboard with a per-session token, key-only cluster access. *Integrity:* every trace file carries a SHA-256 manifest together with the exact configuration, git commit and RNG seeds that produced it, so results cannot be silently altered after the fact. *Secrets:* Hugging Face tokens supplied through environment variables, never committed, with pre-commit secret scanning. Released artefacts are licence-reviewed (Llama 3 Community Licence; Apache-2.0 for Mistral and Qwen) before publication.
Hardware Block Diagram	GPU host block: CPU (8–16 cores) with 32–128 GB system RAM → PCIe 4.0 → CUDA GPU (16–80 GB VRAM) holding model weights and the live KV cache. An asynchronous CPU-side ring buffer receives flushed attention statistics and writes to local NVMe scratch, which is periodically synchronised to institutional or NAS storage for the released dataset. Shown as the Instrumented Model Runtime and Trace Store layers of Figure 1.

Design Component	Details / Reference
Deployment Architecture	Reproducible environment as a Conda lock file plus a CUDA-base Dockerfile. Experiments launched as SLURM array jobs, one array index per (model, arm, pressure, seed) cell, with checkpoint and resume per conversation; Google Colab Pro and Kaggle are the documented fallback path using 4-bit quantization. Release targets: code on GitHub under MIT, trace dataset on the Hugging Face Hub with a versioned schema and a Zenodo DOI, dashboard runnable via `docker compose up`.
Other Design Artifacts	Formal metric specification (Section 6) with reference implementations and unit tests on synthetic caches of known structure; an experiment configuration schema (Hydra/OmegaConf) that fully determines a run; an intervention specification covering the target-set grammar, modes and budgets; a pre-registered analysis plan fixing thresholds, multiple-comparison corrections and negative controls *before* the main runs, to prevent post-hoc selection; and a model and data card shipped with the released dataset.

7 Algorithms / Methodology

Proposed Algorithm / Methodology Description

The methodology treats the Key–Value cache as a measurable, editable object and proceeds in four stages: instrument, measure, attribute, intervene.

Stage 1 — Instrument. A frozen model is executed with a tracing cache and per-head attention hooks. For a conversation of T tokens on a model with L layers and H heads, the runtime memory after step T is $\mathcal{M}_T = \{(k_{\ell,h,t}, v_{\ell,h,t})\}$, and every entry carries provenance $p_{\ell,h,t} = (\ell, h, t, \tau(t), a(t), c(t))$ — layer, head, absolute position, turn, agent and source channel. This provenance is the contribution that makes the later attribution possible; existing KV work records none of it.

Stage 2 — Measure. Four metrics characterise how long and how strongly an entry keeps influencing generation. Writing $\bar{\alpha}_{q,t} = \frac{1}{LH} \sum_{\ell,h} \alpha_{q,t}^{\ell,h}$ for the head-averaged attention from query q to key t, the metrics are:

KV reuse score, normalised so that a token attended exactly as much as a uniform baseline

scores 1:

$$R(t) = \frac{1}{T-t} \sum_{q=t+1}^{T} q\, \bar{\alpha}_{q,t}.$$

Memory lifetime, the last horizon at which smoothed attention still exceeds a relative threshold κ/q over a window w:

$$L(t) = \max\left\{ q - t \;\middle|\; \tilde{\alpha}_q(t) > \frac{\kappa}{q} \right\}, \qquad \tilde{\alpha}_q(t) = \frac{1}{w} \sum_{j=q-w+1}^{q} \bar{\alpha}_{j,t}.$$

Attention persistence, the fraction of (query, head) pairs for which t stays in the top-k of the attention distribution:

$$P(t) = \frac{1}{(T-t)LH} \sum_{q>t} \sum_{\ell,h} \mathbf{1}\left[\, t \in \mathrm{TopK}_k\big(\alpha^{\ell,h}_{q,\cdot}\big) \right].$$

Centrality, from the directed attention graph $G = (V, E)$ with V the token positions and edge weight $\bar{\alpha}_{q,t}$, computed by PageRank with damping d:

$$c(t) = \frac{1-d}{T} + d \sum_{q} \frac{\bar{\alpha}_{q,t}}{\sum_{t'} \bar{\alpha}_{q,t'}}\, c(q).$$

A position is declared a **Memory Attractor** when all three conditions hold jointly,

$$\mathcal{A} = \left\{ t \;\middle|\; z_R(t) > z^\star \;\wedge\; L(t) > \lambda T \;\wedge\; P(t) > p^\star \right\},$$

where z_R is the within-conversation z-score of R. Early positions ($t \le 4$) are reported as a *separate baseline class*, because they are the already-known attention-sink phenomenon; only structures outside that class count as a new finding.

Stage 3 — Attribute. For each head, value vectors at attractor positions are pooled per conversation n, $u^n_{\ell,h} = \frac{1}{|\mathcal{A}_n|} \sum_{t \in \mathcal{A}_n} v_{\ell,h,t}$, and a cross-validated logistic probe $\sigma(w_{\ell,h}^\top u + b)$ is fitted to predict the conversation's bias polarity y_n. Predictive evidence alone is not enough, so the **Bias Reservoir** score multiplies held-out AUC by the measured effect of ablating that head:

$$S(\ell, h) = \mathrm{AUC}_{\ell,h} \cdot \max\left(0,\; \frac{B - B^{\mathrm{abl}(\ell,h)}}{B}\right).$$

Reservoirs are the top-m heads by S that survive Benjamini–Hochberg correction against a random-head null. The bias index $B \in [-1, 1]$ instantiates to the BBQ ambiguous- and disambiguated-context bias scores, the StereoSet stereotype score, and the CrowS-Pairs

pseudo-log-likelihood preference rate.

Memory Contagion is defined so that it cannot be explained by exchanged text. With three arms — shared-KV (S), text-only (T) and isolated control (C) — the memory-mediated and text-mediated components are separated as

$$\mathrm{MC}_{i\to j} = B_j^{(S)} - B_j^{(T)}, \qquad \mathrm{TC}_{i\to j} = B_j^{(T)} - B_j^{(C)}.$$

Shared-KV transfer is only defined between agents instantiating the *same* base model, with RoPE positions re-indexed on transplant; this is a deliberate scope restriction, since cache entries are not portable across tokenizers or architectures. Directed dependence is corroborated by transfer entropy between the agents' per-turn bias series and by attractor-set overlap $J = |\mathcal{A}_i \cap \mathcal{A}_j|/|\mathcal{A}_i \cup \mathcal{A}_j|$.

Memory phase transitions are tested by sweeping semantic pressure $\pi \in [0, 1]$ and fitting

$$B(\pi) = B_0 + \frac{B_1 - B_0}{1 + e^{-\kappa(\pi - \pi_c)}}, \qquad \chi(\pi) = \frac{dB}{d\pi},$$

with π_c the critical pressure and χ the susceptibility. A transition is claimed only if the logistic fit beats a linear null by AIC and BIC, the confidence interval on κ excludes the linear regime, and an independent changepoint detector locates a break inside the confidence interval of π_c for at least three seeds and two models.

Stage 4 — Intervene. Three KV-only edits are applied during a seed-matched replay: *ablation* sets the pre-softmax logit of every target entry to $-\infty$ and renormalises; *perturbation* projects the probe-identified bias direction out of the target value vectors, $v \leftarrow v - \epsilon\,(\hat{w}^\top v)\,\hat{w}$; *gating* rescales attention to the target set by $g \in [0, 1]$ before renormalising. The causal effect is the average difference under intervention, $\mathrm{ACE}(S) = E[B \mid do(S)] - E[B]$, estimated by paired replay under fixed seeds and compared against random-head and sink-only negative controls. An intervention is accepted only if it also satisfies the coherence budget: perplexity degradation $\leq 5\%$ and MMLU/GSM8K subset accuracy within 2 points of baseline. Since no gradient is ever taken and no parameter is written, the entire method runs at inference cost.

Pseudocode / Mathematical Formulation (Optional)

Algorithm 1 — Instrumented multi-agent conversation (Module 1)

```
Input :  model M, agents A, probe p, arm, pressure pi, turns T, seed s
Output:  trace D, bias series B, coherence series C
  1 setAllSeeds(s); cache <- TracingCache(M.layers, M.kvHeads)
  2 attach eager-attention hooks on every attention module of M
  3 ctx <- SeedContext(p, pi); D <- {}
  4 for tau = 1 to T do
    5 a <- Schedule(A, tau)
    6 if arm = SHARED then cache <- Merge(cache, KV[peer], reindexRoPE)
    7 y, cache <- M.generate(ctx, cache, returnAttentions = True)
    8 for each (l, h, t) written this turn do
      9 D <- D + {(l, h, t, tau, a, channel, attnMass, kNorm, vNorm)}
    10 if sampled(tau) then dumpRawKV(cache, tau)
    11 B[tau] <- BiasIndex(y, p); C[tau] <- Coherence(y)
    12 ctx <- Append(ctx, y)
  13 flushAsync(D); writeManifest(config, gitCommit, s)
  14 return D, B, C
```

Algorithm 2 — Attractor detection and reservoir localization (Modules 2–3)

```
Input :  traces D, bias labels y, thresholds zStar, lambda, pStar, m
Output:  attractor sets Aset, reservoir set Res
  1 for each conversation n and position t do
    2 compute R(t), L(t), P(t) from the metric definitions above
  3 G <- AttentionGraph(D); c <- PageRank(G, d = 0.85)
  4 Aset[n] <- { t :  z(R(t)) > zStar and L(t) > lambda*T and P(t) > pStar }
  5 split Aset[n] into SINK (t <= 4) and NOVEL (t > 4); report separately
  6 for each head (l, h) do
    7 U <- { mean of v(l,h,t) over t in NOVEL[n] :  for all n }
    8 AUC(l,h) <- CrossValLogisticProbe(U, y, folds = 5)
    9 Babl <- ReplayWithAblation(head = (l,h))
    10 S(l,h) <- AUC(l,h) * max(0, (B - Babl) / B)
  11 Res <- top-m heads by S surviving BH-FDR vs.  random-head null
  12 return Aset, Res
```

Algorithm 3 — Causal KV-only intervention and validation (Module 4)

```
Input :  conversation seed s, target set S, mode in {ablate, perturb, gate}
Output:  ACE, dPPL, dAcc, accept/reject
  1 setAllSeeds(s); Bbase, PPLbase, AccBase <- Replay(s, intervention = none)
  2 for each replay step q do
    3 if mode = ablate :  logit(q,t) <- -inf for all t in S, then renormalize
    4 if mode = perturb :  v(l,h,t) <- v - eps*(wHat' v)*wHat for all t in S
    5 if mode = gate :  alpha(q,t) <- normalize(g[t] * alpha(q,t))
  6 Bint, PPLint, AccInt <- Evaluate(replayed conversation)
  7 ACE <- Bbase - Bint; dPPL <- (PPLint - PPLbase)/PPLbase
  8 Bnull <- Replay with |S| random non-reservoir heads // negative control
  9 Bsink <- Replay with sink-only entries // negative control
  10 accept if ACE exceeds both controls at p < 0.05 (bootstrap, 5 seeds x 3 models)
    11 and dPPL <= 0.05 and dAcc <= 2 points
  12 return ACE, dPPL, dAcc, decision
```

8 Technology Stack

Category	Version	Technology / Tool
Programming Language	3.11	Python (primary implementation language); SQL (SQLite/DuckDB dialect) for trace queries; Bash for cluster job scripts
Framework	2.5 / 4.46 / 1.0	PyTorch (CUDA 12.1 build) as the tensor and hook layer; Hugging Face Transformers, whose `DynamicCache` is subclassed by `TracingCache`; Hugging Face Accelerate for device placement and offloading
Frontend	1.39 / 5.24	Streamlit for the researcher dashboard; Plotly for interactive attention heatmaps, attractor-lifetime timelines and reservoir maps; Matplotlib and Seaborn for publication figures
Backend	0.115 / 0.32	FastAPI read-only trace-query service with an auto-generated OpenAPI schema; Uvicorn ASGI server bound to localhost
Database	3.45 / 17.0 / 1.1	SQLite as the metadata and conversation index; Apache Parquet via PyArrow as the columnar store for the `kvRecord` fact table; DuckDB for analytical queries over Parquet without loading into memory
Libraries	see software table	`transformers`, `torch`, `accelerate`, `bitsandbytes` (NF4 quantization), `datasets`, `numpy`, `scipy`, `scikit-learn` (logistic probes, AUC, cross-validation), `networkx` (attention-graph centrality), `ruptures` (changepoint detection), `statsmodels`, `pandas`, `pyarrow`, `hydra-core` (experiment configs), `tqdm`, `pytest`
IDE / Development Environment	1.9x / 4.2	Visual Studio Code with the Python, Jupyter and Remote-SSH extensions; JupyterLab for exploratory trace analysis on the GPU node
Version Control	2.43 / 3.5	Git with GitHub for code and issue tracking; Git LFS for figures and small fixtures; DVC for versioning the generated trace dataset

Category	Version	Technology / Tool
Cloud / Deployment Platform	—	Institute GPU cluster scheduled through SLURM (primary); Google Colab Pro and Kaggle Notebooks as the documented low-VRAM fallback; Hugging Face Hub and Zenodo for public release of the dataset; Docker for a reproducible runtime image
Operating System	22.04 LTS	Ubuntu Linux on the compute node; Windows 11 with WSL2 for local development
Other Tools	—	Overleaf for report preparation; Turnitin for the originality check; Zotero for IEEE-style reference management; `ruff` and `black` for linting and formatting; GitHub Actions for continuous integration; Weights & Biases (optional, offline mode) for run tracking

9 Hardware and Software Requirements

Requirements are stated at two tiers. The **minimum** tier is what is needed to execute the full method at reduced scale (4-bit quantized weights, shorter contexts, fewer seeds); the **recommended** tier is what is needed to produce the complete released dataset across all three models at full precision.

Hardware Requirements

Component	Specification
Processor	Minimum: 8-core x86-64 (Intel Core i7 / AMD Ryzen 7). Recommended: 16-core or higher Xeon / EPYC cluster node. The CPU handles asynchronous trace serialisation and Parquet compression while the GPU generates, so core count directly affects the instrumentation overhead.
RAM	Minimum: 32 GB — sufficient because attention statistics are reduced on-GPU and streamed out in batches rather than held whole. Recommended: 64–128 GB, which allows raw KV dumps to be staged in a host-side ring buffer without stalling generation.

Component	Specification
Storage	Minimum: 500 GB SSD. Recommended: 2 TB NVMe scratch plus institutional or NAS backup. Provenanced statistics cost roughly 8–16 MB per conversation, so the full corpus of statistics is tens of gigabytes; the 5% raw-tensor sample dominates the remainder, since uncompressed KV state costs about 130 KB per token for an 8B model in bf16.
GPU (If Applicable)	Minimum: single CUDA GPU with 16 GB VRAM (NVIDIA T4, RTX 4060 Ti 16 GB, or Colab equivalent), running 7–8B models under 4-bit NF4 quantization with batch size 1 and contexts capped near 2k tokens. Recommended: NVIDIA A100 40/80 GB or H100, giving bf16 precision, full-length contexts, eager-attention capture without truncation, and all three models in one sweep. Compute capability ≥ 7.5 is required for the quantization kernels.
Internet Connectivity	Required for the initial download of model weights (roughly 15–16 GB per model in bf16) and the four benchmark datasets, and for artefact upload at release. A stable ≥ 10 Mbps link is sufficient; experiments themselves run fully offline.
Other Hardware	1 TB external or NAS storage for archiving the released dataset; UPS-backed power or SLURM checkpointing for multi-hour sweeps; a development workstation for dashboard use and report preparation.

Software Requirements

Software	Version / Details
Operating System	Ubuntu 22.04 LTS (primary target for the GPU node); Windows 11 with WSL2 Ubuntu for local development. No Windows-native CUDA path is required.
Programming Language(s)	Python 3.11.x (3.10 minimum); SQL for the trace index; Bash for SLURM submission scripts.
Framework(s)	PyTorch 2.5 with CUDA 12.1; Hugging Face Transformers 4.46; Accelerate 1.0; FastAPI 0.115 with Uvicorn 0.32; Streamlit 1.39.

Software	Version / Details
Database	SQLite 3.45 for conversation and run metadata; Apache Parquet via PyArrow 17 for the KV record store; DuckDB 1.1 (optional) for analytical queries directly over Parquet.
IDE / Editor	Visual Studio Code 1.9x with Python, Jupyter and Remote-SSH extensions; JupyterLab 4.2 on the compute node; Overleaf for the report.
Libraries / Packages	`numpy` 2.0, `scipy` 1.14, `pandas` 2.2, `scikit-learn` 1.5, `networkx` 3.3, `ruptures` 1.1, `statsmodels` 0.14, `datasets` 3.0, `bitsandbytes` 0.44, `pyarrow` 17.0, `duckdb` 1.1, `plotly` 5.24, `matplotlib` 3.9, `seaborn` 0.13, `hydra-core` 1.3, `tqdm` 4.66, `pytest` 8.3, `ruff`, `black`. All versions pinned in a lock file.
Other Dependencies	NVIDIA driver $\geq$ 535 with CUDA Toolkit 12.1 and cuDNN 8.9; Git 2.43 with Git LFS; Docker 27 (optional, for the reproducible image); a Hugging Face account with the Llama 3 Community Licence accepted, required before the Llama weights can be downloaded; Turnitin access for the originality check.

10 Design Considerations (Non-Functional)

Design Aspect	Description
Scalability	The trace schema separates cheap provenanced statistics, retained for every run, from expensive raw KV tensors, retained only for a stratified sample — this is what keeps a corpus of a thousand multi-turn conversations within a few hundred gigabytes instead of tens of terabytes. Parquet files are sharded by (model, conversation) so analysis parallelises without coordination, and writes are streamed rather than buffered whole. Experiments are embarrassingly parallel across the (model, arm, pressure, seed) grid and run as SLURM array jobs. The same code path degrades to a single 16 GB GPU by switching a configuration flag to 4-bit weights and a shorter context, with no source changes.

Design Aspect	Description
Performance	The central risk is that instrumentation distorts the phenomenon being measured. The design targets under 2× wall-clock overhead relative to uninstrumented generation, achieved by accumulating attention statistics on-GPU in reduced precision, flushing asynchronously on a background thread, and using eager attention only on turns where weights must be materialised. Measured overhead is reported as an experimental control rather than assumed. Analysis runs offline over Parquet, so dashboard queries never contend with generation.
Security	The threat model is research integrity, not adversarial attack. Dependencies are pinned and hash-verified with `pip-audit` in CI; model downloads are checksum-verified; the dashboard binds only to `127.0.0.1` behind a per-session token and is reached over SSH, with no service exposed to the campus network; Hugging Face tokens are supplied through environment variables with pre-commit secret scanning. Every trace file carries a SHA-256 manifest recording the configuration, git commit and RNG seeds that produced it, so a result cannot be silently altered after publication.
Reliability	Every run is deterministic and replayable: all RNG sources are seeded and recorded, and CUDA deterministic algorithms are enabled where available — without this, seed-matched replay would be meaningless and no causal claim could be made. Long sweeps checkpoint and resume per conversation. Each metric has unit tests against synthetic caches with analytically known attractor structure, so implementation errors are caught before they reach experiments. Every reported effect is validated across 3 models and 5 seeds with bootstrap confidence intervals and against random-head and sink-only negative controls.
Maintainability	The four modules are separate packages with typed interfaces, so analysis can be rerun without regenerating traces. All model-specific behaviour — attention module naming, GQA head mapping, RoPE re-indexing — is isolated behind a single `ModelAdapter` interface, so supporting a new architecture means writing one adapter file rather than editing the pipeline. Experiments are fully described by Hydra configuration files with no hard-coded paths. CI runs `pytest` and `ruff` on every commit, and the released dataset schema is semantically versioned.

Design Aspect	Description
Privacy	The project involves no human subjects and collects no personal data. All prompts are either synthetic or drawn from public, peer-reviewed benchmarks, and the released traces contain token identifiers and numeric statistics derived from that public benchmark text only — no user conversations, no scraped content. Before release, sampled raw KV dumps are screened to confirm they cannot be used to reconstruct verbatim material beyond the already-public source text.
Ethical Considerations (If Applicable)	The work deliberately elicits biased model output in order to study it. Such output is quarantined inside the trace store, never surfaced as model recommendations, and released only in the aggregated, labelled form needed for reproduction, accompanied by a model and data card. The dual-use risk is explicit and acknowledged: a mechanism that steers behaviour by editing the KV cache could in principle amplify bias as easily as suppress it. This is mitigated by releasing detection, measurement and evaluation code alongside the intervention toolkit, by shipping an explicit intended-use statement with the release, and by requiring guide review before any public artefact is published. Reporting is pre-registered so that negative findings are published rather than discarded.

11 Expected Outcome

The project is expected to deliver the first end-to-end account of the KV cache as a behaviourally consequential runtime memory, together with the software and data needed to reproduce and extend that account. Concretely:

1. **Instrumentation framework (Module 1).** An open-source Python/PyTorch library that wraps any Hugging Face decoder-only model and emits fully provenanced KV traces at under 2× uninstrumented decoding latency, validated on Llama-3-8B-Instruct, Mistral-7B-Instruct and Qwen2.5-7B.
2. **Evidence on Memory Attractors (Module 2).** A quantitative characterisation of persistent KV structures *beyond* the known attention-sink and heavy-hitter classes, with reuse, lifetime, persistence and centrality distributions reported per model and per bias category.
3. **A Bias Reservoir map (Module 2).** A ranked list of (layer, head) pairs whose

attractor-pooled value vectors predict conversational bias polarity above chance — target held-out AUC ≥ 0.70 — and whose ablation measurably reduces the bias index, establishing that bias accumulation is localized rather than diffusely encoded.

4. **Contagion and phase-transition results (Module 3).** A three-arm comparison that isolates the memory-mediated component of bias transfer from the text-mediated component, plus a fitted bias-versus-pressure curve with an estimated critical pressure π_c and bootstrap confidence interval, tested against a linear null.

5. **Causal control (Module 4).** A demonstration that KV-only interventions shift the bias index by a target $\geq 20\%$ relative reduction while holding perplexity degradation $\leq 5\%$ and MMLU/GSM8K accuracy within 2 points — achieved at inference time, with no parameter updates.

6. **Public artefacts.** An annotated runtime-memory trace dataset ($\geq$ 1,000 conversations across 3 models and 4 benchmarks) with schema documentation and a model card; an interactive dashboard; a causal KV intervention toolkit; and a research paper submitted to ACL/EMNLP, ICLR or CODS-COMAD.

7. **Publishable negative results.** If attractors prove statistically indistinguishable from attention sinks, if reservoirs prove diffuse rather than localized, or if interventions cannot move behaviour within the coherence budget, those outcomes are themselves substantive evidence about the limits of runtime-memory explanations. The experimental design — pre-registered thresholds, negative controls, multi-model replication — is deliberately constructed so that the contribution does not depend on a positive result.

12 References

1. G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis, "Efficient streaming language models with attention sinks," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2024. [Online]. Available: https://arxiv.org/abs/2309.17453

2. Z. Zhang *et al.*, "H2O: Heavy-hitter oracle for efficient generative inference of large language models," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023. [Online]. Available: https://arxiv.org/abs/2306.14048

3. M. Oren, M. Hassid, N. Yarden, Y. Adi, and R. Schwartz, "Transformers are multi-state RNNs," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2024, pp. 18724–18741. [Online]. Available: https://aclanthology.org/2024.emnlp-main.1043

4. E. Todd, M. L. Li, A. S. Sharma, A. Mueller, B. C. Wallace, and D. Bau, "Function vectors in large language models," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2024. [Online]. Available: https://arxiv.org/abs/2310.15213

5. Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, and I. Mordatch, "Improving factuality and reasoning in language models through multiagent debate," in *Proc. 41st Int. Conf. Machine Learning (ICML)*, PMLR vol. 235, 2024, pp. 11733–11763. [Online]. Available: https://arxiv.org/abs/2305.14325

6. I. O. Gallegos *et al.*, "Bias and fairness in large language models: A survey," *Computational Linguistics*, vol. 50, no. 3, pp. 1097–1179, 2024, doi: 10.1162/coli_a_00524.

7. A. Parrish *et al.*, "BBQ: A hand-built bias benchmark for question answering," in *Findings of the Association for Computational Linguistics: ACL*, 2022, pp. 2086–2105. [Online]. Available: https://aclanthology.org/2022.findings-acl.165

8. S. Lin, J. Hilton, and O. Evans, "TruthfulQA: Measuring how models mimic human falsehoods," in *Proc. 60th Annu. Meeting Assoc. Computational Linguistics (ACL)*, 2022, pp. 3214–3252. [Online]. Available: https://aclanthology.org/2022.acl-long.229

Faculty Remarks

Guide Remarks

Project Guide	**Project Coordinator**
Name: Dr Thara Shyam	Name: Dr Divya Udayan J
Signature: _____	Signature: _____